

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



BÁO CÁO DỰ ÁN CÔNG NGHỆ
Ngành: Khoa học máy tính

**NGHIÊN CỨU VÀ PHÁT TRIỂN CÔNG CỤ HỖ TRỢ
XÂY DỰNG TỰ ĐỘNG MÔ HÌNH ĐẶC TẢ YÊU CẦU**

Sinh viên: Lê Thế Phương Minh - 22028089
Môn học: Dự án công nghệ
Lớp học phần: INT3132 2
Giảng viên hướng dẫn: PGS TS Đặng Đức Hạnh

HÀ NỘI - 2025

Tóm tắt

Dự án này nghiên cứu giải pháp tự động hóa chuyển đổi yêu cầu phần mềm sang Ca sử dụng và sơ đồ PlantUML, khắc phục hạn chế về ảo giác của LLM thông qua quy trình *Chain-of-Verification*.

Nghiên cứu đóng góp ba thành phần cốt lõi: (1) Cấu trúc dữ liệu chuẩn hóa GenUseCase; (2) Hệ thống đánh giá điểm định lượng; và (3) Cơ chế phản biện giúp triệt tiêu thiên kiến nhất quán. Kết quả thực nghiệm cho thấy giải pháp giúp nâng cao đáng kể tính chính xác và logic của đặc tả kỹ thuật, đồng thời hình thành quy trình tự hiệu chỉnh hiệu quả cho các hệ thống hỗ trợ phát triển phần mềm.

Mục lục

Mục lục

Danh mục các từ viết tắt

Chương 1 Các định nghĩa nền tảng	2
1.1 Ca sử dụng và Biểu đồ tuần tự	2
1.2 Model Context Protocol	3
1.3 Phương pháp Chain-of-Verification (CoVe)	4
Chương 2 Phương pháp luận	7
2.1 GenUseCase	7
2.2 Đảm bảo chất lượng UseCase bằng CoVe+	9
2.2.1 Tổng quan	9
2.2.2 Sinh phản hồi cơ sở	10
2.2.3 Thực hiện tính điểm bằng hàm tính điểm	11
2.2.4 Lập kế hoạch xác minh	13
2.2.5 Thực thi xác minh	13
2.2.6 Sinh phản hồi xác minh cuối cùng	14
2.3 Xây dựng các tool	14
Chương 3 Thử nghiệm	16
3.1 Bộ dữ liệu	16
3.2 Phương pháp đánh giá	16
3.3 Các Mô hình được sử dụng	17
3.4 Kết quả thử nghiệm	18
3.5 Nhận xét	18
Chương 4 Kết luận	20
Tài liệu tham khảo	22

Danh sách hình vẽ

1.1	Biểu đồ Ca sử dụng Rút tiền mặt	3
2.1	Biểu đồ Metamodel của GenUseCase	8

Danh sách bảng

1.1	Đặc tả Ca sử dụng Rút tiền mặt	6
3.1	Kết quả đánh giá so sánh trung bình giữa Ca sử dụng Cơ sở và Ca sử dụng Cải thiện trên thang điểm 10	19

Danh mục các từ viết tắt

STT	Từ viết tắt	Cụm từ đầy đủ	Cụm từ tiếng Việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	ML	Machine Learning	Học máy
3	DL	Deep Learning	Học sâu
4	SOTA	State-of-the-art	Các mô hình hiện đại nhất hiện tại
5	S.O.	Structured Output	Đầu ra có cấu trúc
6	LLM	Large Language Model	Mô hình ngôn ngữ lớn

Mở đầu

Sự phát triển của công nghệ phần mềm hiện đại khiến việc mô hình hóa yêu cầu ngày càng phức tạp. Dù các mô hình ngôn ngữ lớn (LLM) mở ra hướng đi mới trong việc tự động hoá chuyển đổi yêu cầu, chúng vẫn đối mặt với thách thức lớn về tính nhất quán và độ chính xác (ví dụ: các lỗi ảo giác). Dự án này tập trung phát triển một phương pháp mô hình hóa mới dựa trên cơ chế tự sửa lỗi của LLM thông qua phương pháp *Chain-of-Verification*.

Để giải quyết các hạn chế về mặt cấu trúc dữ liệu Use Case đầu ra của LLM, dự án đề xuất cấu trúc mới: **GenUseCase**: Một cấu trúc phẳng được thiết kế riêng cho đầu ra của LLM. GenUseCase loại bỏ các vòng đệ quy phức tạp để tránh lỗi "ảo giác đệ quy" thường gặp, giúp nội dung sinh ra ổn định hơn.

Cấu trúc báo cáo này bao gồm: Chương 1 trình bày lý thuyết nền tảng, Chương 2 mô tả các phương pháp và lý thuyết đề xuất, Chương 3 trình bày các thực nghiệm và kết quả đánh giá, và Chương 4 rút ra kết luận và hướng phát triển tương lai.

Chương 1

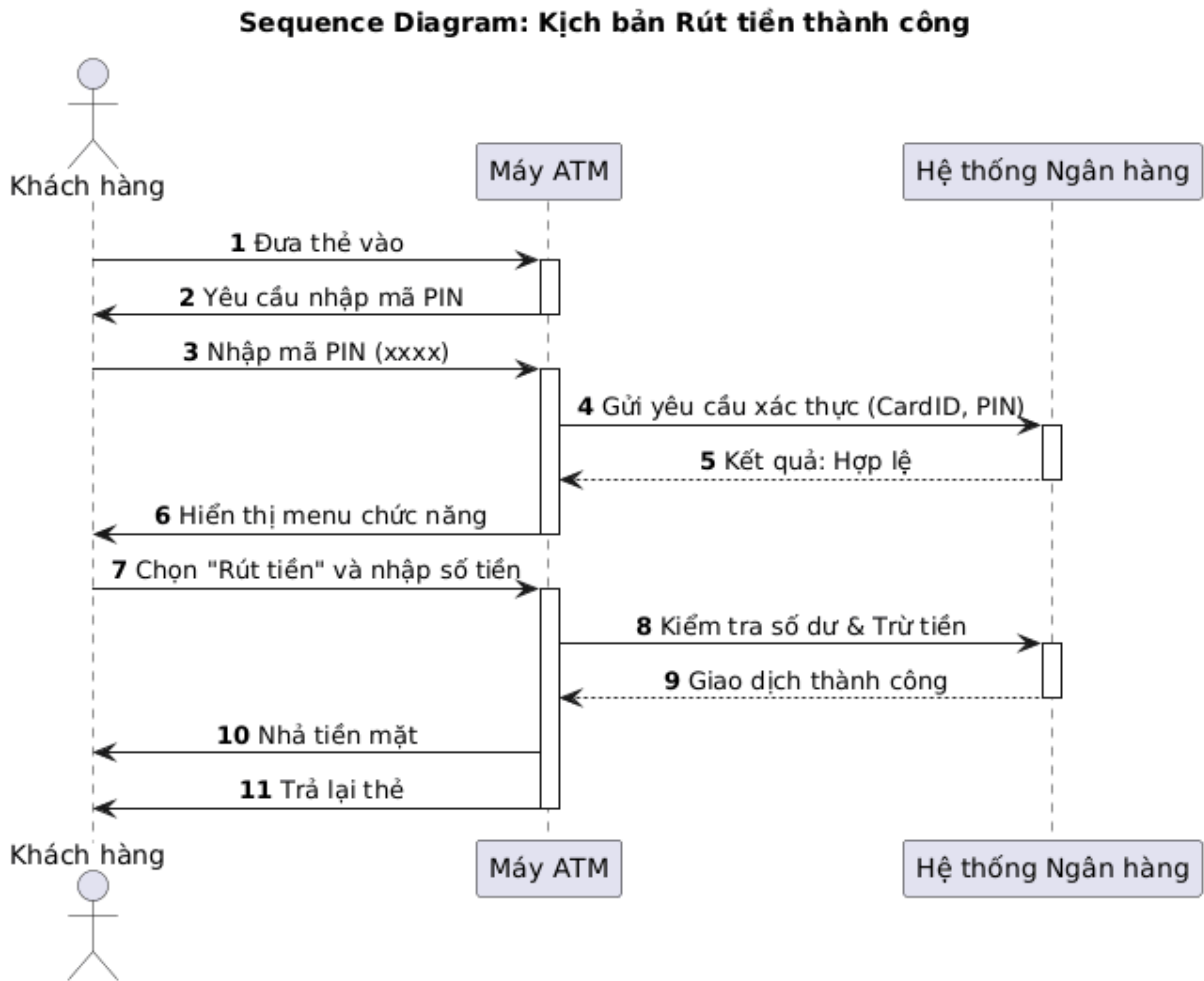
Các định nghĩa nền tảng

1.1 Ca sử dụng và Biểu đồ tuần tự

Trong kỹ thuật phần mềm, Ca sử dụng (Use Case) là một kỹ thuật mô tả các yêu cầu chức năng của hệ thống thông qua sự tương tác giữa tác nhân (actor) và hệ thống nhằm đạt được một mục tiêu nghiệp vụ cụ thể. Mỗi Use Case thể hiện một kịch bản sử dụng điển hình, phản ánh cách hệ thống được kỳ vọng hoạt động từ góc nhìn của người dùng hoặc các hệ thống bên ngoài.

Một Ca sử dụng riêng lẻ thường được đặc tả theo cấu trúc chuẩn, bao gồm các thành phần chính như: các tác nhân tham gia, tiền điều kiện, hậu điều kiện, luồng chính, các luồng thay thế hoặc ngoại lệ, và các bước xử lý đơn vị. Cấu trúc này giúp mô tả đầy đủ bối cảnh, điều kiện và hành vi của hệ thống khi thực hiện một chức năng cụ thể, đồng thời làm rõ các trường hợp xử lý khác nhau có thể xảy ra.

Use Case có thể được biểu diễn dưới nhiều hình thức nhằm phục vụ các mục đích khác nhau trong quá trình phát triển phần mềm. Đặc tả dạng văn bản cho phép mô tả chi tiết yêu cầu chức năng bằng ngôn ngữ tự nhiên, dễ hiểu đối với các bên liên quan. Trong khi đó, Biểu đồ tuần tự (Sequence Diagram) cung cấp cái nhìn động và kỹ thuật hơn, mô tả cách các đối tượng trong hệ thống tương tác với nhau theo trình tự thời gian để hiện thực hóa một kịch bản cụ thể của Use Case. Việc kết hợp cả hai hình thức biểu diễn giúp đảm bảo yêu cầu được mô tả rõ ràng, nhất quán và có thể chuyển hóa hiệu quả sang thiết kế và cài đặt hệ thống. Hình 1.1 biểu diễn một ca sử dụng dưới dạng biểu đồ tuần tự, trong khi bảng 1.1 biểu diễn dưới dạng văn bản, thường thấy trong các đặc tả hệ thống.



Hình 1.1: Biểu đồ Ca sử dụng Rút tiền mặt

1.2 Model Context Protocol

Model Context Protocol (MCP) là một tiêu chuẩn giao thức mở được thiết kế nhằm giải quyết bài toán kết nối giữa LLM và nguồn dữ liệu, hay các chức năng mở rộng bên ngoài. Đóng vai trò như một giao diện thống nhất (tương tự khái niệm cổng USB-C cho ứng dụng AI), MCP chuẩn hóa cách thức các hệ thống AI giao tiếp với các nguồn tài nguyên bên ngoài như trên. Thay vì phải xây dựng các trình kết nối riêng lẻ cho từng nguồn dữ liệu và từng LLM, MCP cung cấp một kiến trúc chung giúp giảm thiểu sự phức tạp trong tích hợp, đồng thời đảm bảo khả năng mở rộng và bảo mật cho các ứng dụng trí tuệ nhân tạo.

Về mặt kỹ thuật, MCP vận hành dựa trên kiến trúc Client-Server sử dụng chuẩn

JSON-RPC 2.0. Hệ sinh thái của giao thức được cấu trúc xoay quanh bốn thành phần cốt lõi: Resources (tài nguyên dữ liệu thụ động như file, logs), Tools (các hàm chức năng cho phép AI thực thi hành động), Prompts (các mẫu tương tác được định nghĩa sẵn), và Sampling (cơ chế cho phép Server chủ động khai thác khả năng xử lý của LLM). Sự kết hợp này cho phép xây dựng các AI Agent không chỉ có khả năng "đọc hiểu" dữ liệu mà còn có thể thực hiện các tác vụ phức tạp trong quy trình nghiệp vụ.

Đặc biệt, việc lựa chọn Model Context Protocol là quyết định tối ưu cho bài toán thu thập đặc tả yêu cầu, xuất phát từ sự tương thích giữa bản chất kiến trúc của giao thức này với phương pháp luận 'human-in-the-loop' (con người trong vòng lặp kiểm soát). Trong quy trình kỹ nghệ yêu cầu, nơi sự chính xác và tính xác thực là yếu tố tiên quyết, MCP cung cấp các cơ chế tương tác hai chiều (như sampling và elicitation) cho phép chuyên gia nghiệp vụ liên tục đánh giá, tinh chỉnh và phê duyệt các đề xuất của AI trước khi chốt phương án cuối cùng. Đồng thời, tính chất mở và độc lập nền tảng của MCP cho phép hệ thống linh hoạt tích hợp vào các mô hình SOTA mà không bị ràng buộc vào một nhà cung cấp cụ thể. Điều này giúp tận dụng tối đa năng lực suy luận ngữ nghĩa vượt trội của các mô hình SOTA để phân tích các yêu cầu phức tạp, trong khi vẫn duy trì được sự kiểm soát chặt chẽ về luồng dữ liệu và quy trình nghiệp vụ thông qua giao thức chuẩn hóa.

1.3 Phương pháp Chain-of-Verification (CoVe)

Một trong những thách thức lớn nhất khi ứng dụng LLM vào các tác vụ đòi hỏi độ chính xác cao là hiện tượng "ảo giác" (hallucination) - khi mô hình sinh ra các thông tin có vẻ hợp lý về mặt ngữ nghĩa nhưng lại sai lệch về logic thực tế. Để giải quyết vấn đề này, Dhuliawala và cộng sự [1] đã đề xuất phương pháp *Chain-of-Verification* (CoVe). Đây là một kỹ thuật suy luận tự phản hồi (self-reflection), cho phép mô hình tự đánh giá và hiệu chỉnh các sai sót của chính mình thông qua một quy trình có hệ thống.

Cơ chế hoạt động cốt lõi của CoVe mô phỏng lại quá trình con người tự kiểm tra lại bài làm của mình, được chia thành bốn bước tuần tự:

1. **Sinh phản hồi cơ sở (Generate Baseline Response):** Mô hình nhận truy vấn đầu vào và sinh ra một bản nháp đầu tiên. Tại bước này, mô hình có thể

vẫn chứa các thông tin thiếu chính xác do bản chất thống kê của việc sinh từ.

2. **Lập kế hoạch xác minh (Plan Verifications):** Dựa trên truy vấn gốc và bản nháp vừa tạo, mô hình tự động phân tích và liệt kê một danh sách các câu hỏi kiểm chứng (verification questions). Các câu hỏi này tập trung vào việc truy vấn lại tính xác thực của các mệnh đề quan trọng trong bản nháp.
3. **Thực thi xác minh (Execute Verifications):** Mô hình trả lời các câu hỏi đã lập ra ở bước trước một cách độc lập. Mục đích là để kiểm tra chéo các thông tin mà không bị thiên kiến bởi ngữ cảnh của câu trả lời ban đầu.
4. **Sinh phản hồi xác minh cuối cùng (Generate Final Verified Response):** Mô hình tổng hợp lại thông tin từ bản nháp và kết quả của quá trình xác minh để tạo ra câu trả lời hoàn chỉnh. Các thông tin mâu thuẫn hoặc sai lệch phát hiện được sẽ bị loại bỏ hoặc sửa chữa.

Một đóng góp quan trọng về mặt kỹ thuật của CoVe là cơ chế Tách biệt ngữ cảnh (Factored Verification). Trong các phương pháp tiếp cận thông thường, nếu mô hình thực hiện xác minh ngay trong cùng một ngữ cảnh với câu trả lời sai ban đầu, nó có xu hướng lặp lại lỗi sai đó. CoVe giải quyết việc này bằng cách tách quy trình xác minh (Bước 3) ra khỏi ngữ cảnh của bản nháp ban đầu. Khi trả lời các câu hỏi kiểm chứng, mô hình không "nhìn thấy" câu trả lời sai trước đó, từ đó đảm bảo tính khách quan và độ chính xác cao hơn.

Bảng 1.1: Đặc tả Ca sử dụng Rút tiền mặt

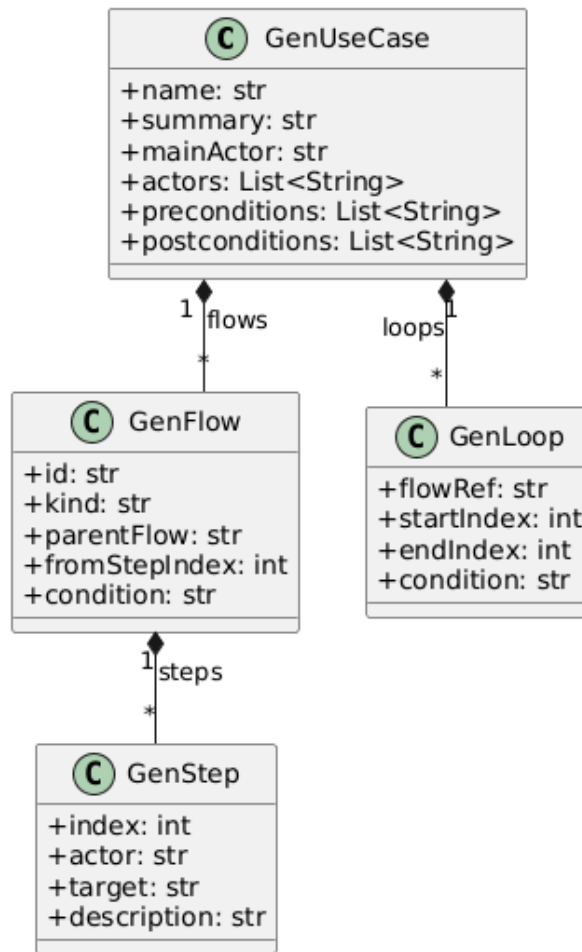
Mục	Nội dung
Tên Ca sử dụng	Rút tiền (Withdraw Cash)
Tác nhân chính	Khách hàng (Customer)
Mô tả ngắn	Khách hàng rút tiền từ tài khoản ngân hàng thông qua máy ATM.
Điều kiện tiên quyết	Máy ATM hoạt động bình thường; thẻ hợp lệ và có thể giao dịch.
Điều kiện hậu	Giao dịch rút tiền được thực hiện thành công; thẻ và tiền được trả lại cho khách hàng.
Luồng sự kiện chính	1. Khách hàng đưa thẻ vào máy ATM. 2. Máy ATM yêu cầu nhập mã PIN. 3. Khách hàng nhập mã PIN. 4. Máy ATM gửi yêu cầu xác thực (CardID, PIN) đến Hệ thống Ngân hàng. 5. Hệ thống Ngân hàng xác thực hợp lệ và phản hồi lại ATM. 6. Máy ATM hiển thị menu chức năng. 7. Khách hàng chọn “Rút tiền” và nhập số tiền cần rút. 8. Máy ATM gửi yêu cầu kiểm tra số dư và trừ tiền đến Hệ thống Ngân hàng. 9. Hệ thống Ngân hàng xác nhận giao dịch thành công. 10. Máy ATM nhả tiền mặt. 11. Máy ATM trả lại thẻ cho khách hàng.
Kết quả	Khách hàng nhận được tiền; tài khoản được cập nhật giao dịch rút tiền.

Chương 2

Phương pháp luận

2.1 GenUseCase

Trong bối cảnh tự động hóa quy trình thu thập yêu cầu bằng LLM, một yêu cầu quan trọng là việc lựa chọn cấu trúc dữ liệu phù hợp để lưu trữ các ca sử dụng. Dự án đã thử nghiệm việc sử dụng một phiên bản thu gọn của FRSL [2], nhưng bản chất cấu trúc FRSL vẫn đặt ra những thách thức đáng kể về độ phức tạp. Các ràng buộc trạng thái (snapshot patterns) và điều kiện OCL chặt chẽ của FRSL thường vượt quá khả năng suy luận nhất quán của LLM, đặc biệt là trong các ca sử dụng phức tạp. Hơn thế nữa, cách liên kết giữa các đơn vị Step cũng khiến cấu trúc này khó có thể được suy luận ra bởi LLM, do chúng có thể mang tính đệ quy nhiều lớp, đặc biệt là khi có chứa các luồng Alt. Chính vì vậy, dự án đề xuất GenUseCase - một biến thể "phẳng" hơn, với mục tiêu là giảm tải các yêu cầu về đặc tả trạng thái hình thức, thay vào đó tập trung tối đa vào việc biểu diễn mạch lạc luồng sự kiện và mối liên kết logic giữa các bước thực hiện. Cách tiếp cận này giúp đảm bảo cấu trúc đầu ra vừa vận với khả năng sinh của LLM mà vẫn giữ được tính tổ chức cần thiết của một ca sử dụng.



Hình 2.1: Biểu đồ Metamodel của GenUseCase

Quản lý Flow bằng tham chiếu phẳng

Trong khi FRSL, hay biến thể đã được đơn giản hoá của nó đã đề cập ở trước, thường mô hình hóa các luồng rẽ nhánh theo cấu trúc cây đệ quy (một bước trong luồng chính chứa trọn vẹn một luồng con), hoặc sử dụng các snapshot để kiểm tra trạng thái của object, trước khi quyết định rẽ nhánh hay rejoin. Để có thể đơn giản hoá việc biểu diễn các luồng này, GenUseCase lựa chọn cách tiếp cận tham chiếu phẳng: thay vì lồng ghép phức tạp, GenUseCase tách các luồng xuất hiện trong một ca sử dụng (Main, các Alternative, các Exception) thành các đối tượng GenFlow độc lập và liên kết chúng thông qua các khóa ngoại (parentFlow, fromStepIndex).

Khoá ngoại parentFlow và fromStepIndex là một khoá ngoại đặc biệt chỉ tồn tại trên các flow không phải main, với mục đích chính là nối một flow con với flow cha của nó. parentFlow biểu thị flow này được nối vào flow cha nào, ngay cả khi flow

cha của nó cũng là con của một flow khác. Điều này giúp đơn giản hoá cách LLM "nhớ" các flow, vì chỉ cần tập trung vào parentFlow và fromStepIndex. Cần lưu ý rằng kiến trúc hiện tại áp dụng cơ chế tự động quay lại: sau khi một luồng phụ kết thúc, luồng điều khiển sẽ mặc định quay về luồng cha, tiếp tục tại bước ngay sau bước fromStepIndex (tức fromStepIndex + 1).

Xử lý Vòng lặp

Một thách thức lớn khác là việc biểu diễn các vòng lặp. Các ý tưởng biểu diễn theo dạng cây (giống với biểu diễn AltFlow) thường coi vòng lặp là một bước riêng biệt chứa các bước con trong nó. Điều này cũng dẫn tới vấn đề giống với xử lý flow, khi mà các luồng trở nên phức tạp, chứa nhiều lớp khác nhau có thể khiến LLM dành quá nhiều chú ý cho việc đảm bảo cấu trúc (hoặc không thể biểu diễn chính xác). Chính vì vậy, GenLoop là một lớp đặc biệt, khi nó biểu diễn một thông tin hỗ trợ cho một luồng GenFlow bất kỳ. Một luồng, khi được đánh dấu bởi GenLoop bằng flowRef, có thể được hiểu là các bước trong khoảng startIndex tới endIndex sẽ lặp với điều kiện là condition. Và cũng như với GenFlow, khi vòng lặp kết thúc, luồng điều khiển tiếp tục tại bước ngay sau bước endIndex, đảm bảo tính "phẳng" của UseCase.

2.2 Đảm bảo chất lượng UseCase bằng CoVe+

2.2.1 Tổng quan

Tổng quan, khoá luận đề xuất một quy trình đảm bảo chất lượng của ca sử dụng được sinh ra, tạm gọi là CoVe+, được phát triển dựa trên CoVe như sau:

1. Sinh phản hồi cơ sở: LLM nhận truy vấn đầu vào và sinh ra một bản nháp đầu tiên, dưới format của GenUseCase.
2. Thực hiện tính điểm ban đầu bằng hàm tính điểm: Hàm này không sử dụng LLM để chấm điểm, mà chỉ thuần túy đưa ra các chỉ số về cấu trúc của ca sử dụng cơ sở. Từ điểm này, hàm cũng sinh ra các chỉ dẫn về thiếu sót bằng văn bản.
3. Lập kế hoạch xác minh: Dựa trên truy vấn gốc, bản nháp vừa tạo, kết quả gợi ý của hàm tính điểm, kết hợp với bộ các câu hỏi trong Checklist: Use Case

[3], LLM sẽ sinh ra các câu hỏi để gợi ý các chỉnh sửa cần thiết của mô hình để đảm bảo các điều kiện trong Checklist.

4. Thực thi xác minh: LLM trả lời các câu hỏi được lập ra ở bước trước một cách độc lập. Mục đích là để không bị thiên kiến bởi ngữ cảnh của câu trả lời ban đầu. [1]
5. Sinh phản hồi xác minh cuối cùng: Mô hình tổng hợp lại thông tin từ bản nháp và kết quả của quá trình xác minh để tạo ra câu trả lời hoàn chỉnh.

Điểm cải tiến then chốt của quy trình này so với CoVe nguyên bản nằm ở sự tích hợp hàm tính điểm (bước 2). Việc này cho phép phát hiện sớm các lỗi cấu trúc và cung cấp gợi ý cải thiện định lượng trước khi bước vào quy trình suy luận phức tạp, giúp bước lập kế hoạch kiểm chứng tập trung sâu hơn vào tính logic và ngữ nghĩa của ca sử dụng.

Ngoài ra, quy trình đề xuất có sự điều chỉnh ở bước thực thi kiểm chứng. Trong khi CoVe gốc ngăn cản bước này tiếp cận phản hồi cơ sở (baseline response), thì với đặc thù của bài toán chuyển đổi yêu cầu thành ca sử dụng, LLM được phép tham chiếu bản nháp đầu tiên. Điều này giúp đảm bảo tính nhất quán của các chi tiết nghiệp vụ đã được trích xuất, đồng thời tập trung vào việc tinh chỉnh thay vì sinh lại từ đầu.

Ở các phần sau, khoá luận sẽ tập trung vào phân tích từng bước trong quy trình.

2.2.2 Sinh phản hồi cơ sở

Ở bước này, sau khi nhận đầu vào từ người dùng, LLM sẽ được prompt để sinh ra một ca sử dụng theo format của GenUseCase đã được đề cập bên trên. Việc prompt được giữ ở mức vừa phải, không yêu cầu LLM phải suy luận ra một bản hoàn chỉnh nhất có thể, mà chủ yếu prompt để yêu cầu LLM bám sát theo định dạng của GenUseCase để có thể ra được một JSON hoàn chỉnh và có ý nghĩa. Đối với các model SOTA hiện tại, có thể sử dụng structured output để yêu cầu format đầu ra giống với yêu cầu của mình.

Mặc dù quy trình suy luận của LLM ở giai đoạn này được giữ ở mức vừa phải, nó vẫn là một bước quan trọng, vì bản thân nó là bản nháp đầu tiên quyết định hướng cải thiện, cũng như là đối tượng so sánh sau khi hoàn thiện bản cuối cùng.

2.2.3 Thực hiện tính điểm bằng hàm tính điểm

Như đã đề cập, hàm tính điểm được thiết kế để đánh giá tính toàn vẹn về mặt cấu trúc của ca sử dụng. Thay vì đi sâu vào ngữ nghĩa nghiệp vụ, hệ thống tập trung đảm bảo kiểm tra ca sử dụng được sinh ra có theo sát định dạng GenUseCase không. Mặc dù có áp dụng phân tích ngữ nghĩa sơ bộ thông qua cơ chế dựa trên quy tắc và từ điển từ khóa, chức năng này đóng vai trò hỗ trợ và chuẩn hóa dữ liệu đầu vào cho bước mô hình hóa, chứ không thay thế hoàn toàn việc thẩm định ý nghĩa của từng bước.

Kiểm tra về mặt cấu trúc

Hàm kiểm tra cấu trúc (`validateUseCaseStructure`) đóng vai trò kiểm tra và ngăn chặn sớm việc xử lý các ca sử dụng không giữ đúng cấu trúc của GenUseCase, hay việc theo cấu trúc nhưng các thành phần trong nó lại rỗng hoặc gây ra các hiện tượng bất lợi. Hàm này kiểm tra các yếu tố chính như sau:

- **Kiểm tra tuân thủ Schema:** Đảm bảo cấu trúc dữ liệu JSON khớp hoàn toàn với định nghĩa GenUseCase (sử dụng Zod).
- **Kiểm tra sự tồn tại của các thành phần cốt lõi:** Các yếu tố bắt buộc để định nghĩa một ca sử dụng hợp lệ:
 1. Luồng chính (MAIN flow) phải tồn tại và không được rỗng.
 2. Các trường thông tin cơ bản (tên, tóm tắt, danh sách tác nhân) phải được khai báo đầy đủ và không rỗng.
- **Kiểm tra tính toàn vẹn của cấu trúc phân cấp các luồng:**
 1. Luồng MAIN phải là duy nhất và không được phép có luồng cha.
 2. Các luồng phụ (Alternative/Exception) bắt buộc phải tham chiếu tới một luồng cha tồn tại.
 3. Hai luồng bất kỳ không được phép trùng lặp định danh.
 4. Không tồn tại tham chiếu vòng tròn giữa các luồng.
- **Kiểm tra tính hợp lệ của tham chiếu:**
 1. Các bước xảy ra rẽ nhánh phải nằm trong phạm vi hợp lệ của luồng cha.

2. Điểm bắt đầu và kết thúc của vòng lặp (nếu có) phải tuân thủ giới hạn chỉ số của luồng chứa nó.

- **Kiểm tra tính liên tục của các bước:** Thứ tự các bước trong mỗi luồng phải là các số nguyên liên tiếp, không được đứt quãng hoặc trùng lặp.
- **Kiểm tra tính nhất quán của tác nhân:** Tác nhân chính (Main Actor) được chỉ định bắt buộc phải nằm trong danh sách các tác nhân đã khai báo.

Khi xảy ra bất kỳ lỗi nào, hàm sẽ trả ngay kết quả, để đảm bảo hệ thống không xử lý thêm. Sau đó, hệ thống có thể route lại để yêu cầu generate lại use case thêm một lần nữa.

Kiểm tra về mặt ngữ nghĩa

Cần xác định rõ rằng hàm kiểm tra này không được thiết kế để 'hiểu' ngữ nghĩa nghiệp vụ hay tính logic thực tế của bài toán như tư duy của con người, mà hoạt động dựa trên giả định: một ca sử dụng có chất lượng cao thường chứa các mẫu câu và từ khóa đặc trưng. Chính từ giả định này, hàm được thiết kế để kiểm tra tần suất xuất hiện của các từ khóa này trong các mô tả của các bước, từ đó cho điểm.

Chi tiết hơn, hàm sử dụng các bộ từ điển định nghĩa sẵn bao gồm các tập từ khóa chức năng như các động từ có ý nghĩa nhập liệu, kiểm tra, hay các từ ngữ chỉ hành động lưu trữ. Từ các từ điển này, thuật toán sẽ quét mô tả của các bước, từ đó rút ra một kết luận đơn giản về ý nghĩa của bước này. Ví dụ, sự hiện diện của các từ như 'validate', 'verify' hay 'check' sẽ được hệ thống ghi nhận là có bước kiểm tra dữ liệu, từ đó cộng điểm cho cấu trúc luồng thay vì thực sự kiểm tra logic kiểm thử đó có đúng hay không.

Kết quả trả về của hàm là một vector, chứa kết quả về các chất lượng có thể dự đoán được bằng giả định trên, từ các chỉ số bao phủ từ khóa, điểm số thành phần của từng loại luồng, cho đến các điểm phạt do vi phạm quy tắc. Kết quả này sau đó sẽ được một hàm khác chuyển hoá thành các kết luận dưới dạng văn bản, có thể đưa vào làm prompt cho bước sau đó.

2.2.4 Lập kế hoạch xác minh

Việc sinh các câu hỏi xác minh một cách độc lập, thay vì tự xác minh và trả lời trong cùng một prompt là một yếu tố cốt lõi trong việc giảm thiểu các ảo giác trong câu trả lời [1]. Đặt trong bài toán về xác minh và đảm bảo chất lượng các ca sử dụng được sinh ra, việc này còn giúp LLM có một "cái nhìn" khác về vấn đề được đặt ra, tránh hiện tượng "thiên kiến nhất quán", xảy ra khi LLM cố gắng bảo vệ hay duy trì câu trả lời mà nó sinh ra từ trước. Ở bước này, LLM được prompt để đóng vai trò là một chuyên gia phân tích, có nhiệm vụ sinh ra các câu hỏi phản biện nhằm mở rộng và hoàn thiện Ca sử dụng. Thay vì đưa ra giải pháp sửa đổi ngay lập tức, LLM được ràng buộc phải đưa ra danh sách các câu hỏi nghi vấn, với số lượng câu hỏi trong nghiên cứu này là 5.

Prompt của LLM trong bước này là tổng hợp của các ý sau:

1. Các câu hỏi trong bộ Checklist [3]
2. Các câu hỏi bổ sung dựa trên kinh nghiệm trong viết ca sử dụng, ví dụ: Nếu có 2 actor về mặt ý nghĩa thì cùng cấp, nhưng lại đóng 2 vai trò khác nhau trong các hành động, thì có cần thiết phải tách ra thành 2 đối tượng khác nhau không?
3. Các phản hồi và gợi ý chỉnh sửa, được trả về sau khi gọi hàm đánh giá.
4. Ca sử dụng được sinh ra ở bước 1.

Bộ câu hỏi Checklist [3] và kinh nghiệm đóng vai trò một baseline đảm bảo Ca sử dụng bao phủ đầy đủ các khía cạnh. Prompt được thiết kế để yêu cầu LLM đối chiếu Ca sử dụng nó nhận được với các yếu tố trong bộ câu hỏi này, đảm bảo LLM có thang đánh giá đúng đắn để kiểm tra ý nghĩa của mô tả các bước, điều mà việc kiểm tra bằng hàm không thực hiện được.

2.2.5 Thực thi xác minh

Dự án sử dụng phương pháp xác minh Factored được CoVe đề xuất, tức là yêu cầu LLM trả lời từng câu hỏi trong một phiên riêng biệt. Cơ chế này giúp loại bỏ sự can thiệp chéo giữa các câu trả lời và triệt tiêu thiên kiến nhất quán mà mô hình thường mắc phải. Tuy nhiên, khác với lý thuyết gốc vốn loại bỏ hoàn toàn dữ liệu

đầu vào để tránh sao chép, nghiên cứu này điều chỉnh quy trình bằng cách đưa bản nháp Ca sử dụng sinh ra ở bước trước vào làm ngữ cảnh tham chiếu cho từng truy vấn, do mục tiêu cốt lõi là đánh giá và hoàn thiện chất lượng đặc tả thay vì kiểm chứng sự kiện đơn thuần. Các câu hỏi sau khi được tách khỏi danh sách tổng hợp sẽ được thực thi song song, cho phép hệ thống xử lý sâu hơn từng vấn đề mà không bị giới hạn bởi độ dài ngữ cảnh chung, dù chi phí tính toán có thể gia tăng. Ngoài việc phản hồi các vấn đề được đặt ra, hệ thống yêu cầu mô hình cung cấp thêm gợi ý cải thiện chi tiết cùng với bản mẫu của các phân đoạn đã được sửa đổi. Việc này đóng vai trò quan trọng trong việc cụ thể hóa ngữ cảnh cho bước xử lý kế tiếp, giúp khoanh vùng chính xác các điểm thiếu sót và nâng cao hiệu quả của quá trình hoàn thiện văn bản.

2.2.6 Sinh phản hồi xác minh cuối cùng

Đây là bước cuối cùng, khi LLM được prompt bằng tổng hợp các cặp câu hỏi - câu trả lời sinh ở bước trước, cùng với phản hồi cơ sở. Đầu ra nhận được sau đó sẽ được chạy lại hàm tính điểm ở bước 2, để đưa ra kết quả định lượng sau quá trình xác minh. Mở rộng ra, hoàn toàn có thể tái kích hoạt bước 3 để có thể sinh câu hỏi cải thiện, lần này là lấy cải thiện của người dùng.

2.3 Xây dựng các tool

Với các bước trong phương pháp CoVe+ đã được đề cập ở trên, dự án đã xây dựng thành các tool riêng biệt để mô hình SOTA hỗ trợ MCP có thể chủ động trong việc gọi, và người dùng cũng có thể điều phối thủ công.

Hệ thống đăng ký 3 công cụ chính vào MCP Server, tương ứng với luồng cải thiện ca sử dụng:

extractUseCase (Trích xuất yêu cầu): Đây là điểm khởi đầu của luồng xử lý (Step 1). Công cụ này tiếp nhận mô tả ngôn ngữ tự nhiên thô từ người dùng, sử dụng khả năng phân tích ngữ nghĩa của LLM để chuẩn hóa thành định dạng JSON có cấu trúc. Đầu ra của công cụ này là nền tảng dữ liệu cho toàn bộ các bước xử lý sau, đảm bảo tính nhất quán về cú pháp.

validateUseCase (Kiểm định chất lượng): Công cụ này có 2 nhiệm vụ chính:

thực hiện tính điểm của một Use Case (Bước 2). Nó tiếp nhận bản nháp từ extractUseCase và chạy hàm tính điểm, nếu điểm số dưới ngưỡng an toàn (< 70) hoặc có lỗi về cấu trúc, sẽ kích hoạt cơ chế gợi ý cải thiện (Bước 3). Nếu điểm số ở trên ngưỡng an toàn, sẽ trả về cho người dùng tùy chọn có muốn cải thiện thêm không.

improveUseCase (Cải thiện Use Case): Tương ứng với bước 4 và bước 5. Công cụ này nhận đầu vào là các câu hỏi từ bước validate, tự động sinh câu trả lời để làm rõ các điểm mơ hồ, sau đó tái cấu trúc lại ca sử dụng. Quá trình này tạo ra một vòng lặp phản hồi kín, giúp nâng cao chất lượng đầu ra mà không cần sự can thiệp thủ công ngay lập tức.

Chương 3

Thử nghiệm

3.1 Bộ dữ liệu

Bộ dữ liệu về các ca sử dụng được đưa vào làm tham chiếu để kiểm tra hiệu quả trong cải thiện các ca sử dụng được trích từ các ca sử dụng trong cuốn *Writing Effective Use Cases* của tác giả Alistair Cockburn.

Bộ dữ liệu bao gồm 17 ca sử dụng, mỗi ca gồm:

- **Mô tả cơ sở (Input):** Yêu cầu đầu vào thô.
- **Ca sử dụng văn bản (Textual Use Case):** Mô tả chi tiết theo định dạng truyền thống.
- **Ca sử dụng chuẩn hóa (GenUseCase Ground Truth):** Phiên bản ca sử dụng mục tiêu được mô hình hóa dưới dạng GenUseCase để làm căn cứ so sánh.

3.2 Phương pháp đánh giá

Để đánh giá mức độ cải thiện của phương pháp, dự án này sử dụng một quy trình chấm điểm như sau:

1. Bước 1 - sinh dữ liệu: Với mỗi yêu cầu đầu vào của ca sử dụng trong bộ dữ liệu, có thể sinh ra 2 phiên bản: ca sử dụng cơ sở, và ca sử dụng được cải thiện thông qua quy trình
2. Bước 2 - so sánh đối chiếu: Một LLM độc lập sẽ đóng vai trò giám khảo để so sánh lần lượt 2 phiên bản trên với ca sử dụng chuẩn (Ground Truth). Đầu vào cho LLM giám khảo bao gồm: Bản ghi yêu cầu gốc, Ca sử dụng cần chấm, và Ca sử dụng chuẩn.

Đầu ra của LLM giám khảo dựa trên thang điểm được đánh giá dựa theo các tiêu chí :

- **Độ bao phủ về ngữ nghĩa:** Đánh giá mức độ tương đồng về mặt ý nghĩa của các bước thực hiện giữa phiên bản sinh ra và phiên bản chuẩn.
- **Độ tương thích về tác nhân:** Kiểm tra sự tương ứng của các Actor. Tiêu chí này tập trung vào vai trò và chức năng của tác nhân trong hệ thống hơn là sự trùng khớp chính xác về tên gọi.
- **Độ bao phủ về các luồng:** Đánh giá khả năng bao quát các luồng đi (luồng chính, luồng rẽ nhánh, luồng ngoại lệ) và sự phân chia logic của chúng so với ca sử dụng gốc.
- **Độ chính xác về logic:** Các bước trong các luồng có ý nghĩa hay không, có bước quan trọng nào đã bị bỏ qua hoặc thêm vào mà không đúng với ca dữ liệu gốc không? Tất nhiên, vẫn cần bám sát vào yêu cầu đầu vào chứ không phải chỉ so sánh 2 ca sử dụng với nhau, vì đôi khi LLM có thể tư duy logic khác hơn.

Việc sử dụng LLM làm giám khảo đánh giá ngữ nghĩa đã được chứng minh là đạt hiệu quả tương đương con người trong các tác vụ tương tự [4]. Phương pháp này đảm bảo khả năng đánh giá đa chiều: vừa so sánh độ khớp với dữ liệu mẫu, vừa kiểm tra tính logic so với yêu cầu đầu vào.

3.3 Các Mô hình được sử dụng

Dự án này sử dụng 3 LLM chính: Claude Sonnet 4.5, Gemini-2.5-flash và GPT-4o-mini.

- Claude Sonnet 4.5: Mô hình điều phối, gọi các tool đại diện cho các chức năng cần thiết trong luồng hoạt động. Hiện tại, đây là mô hình SOTA có sự cân bằng trong khả năng phân tích ý định của người dùng để gọi các function/tool tương ứng, cũng như chi phí hoạt động.
- Gemini-2.5-flash: Mô hình chính, hoạt động xuyên suốt trong các luồng: từ sinh phản hồi cơ sở, tạo và trả lời câu hỏi, cũng như đưa ca sử dụng sau cùng. Gemini-2.5-flash được lựa chọn nhờ sự tối ưu giữa tốc độ xử lý, chi phí và khả năng duy trì ngữ cảnh dài, đảm bảo tính nhất quán cho các tác vụ sinh văn bản khối lượng lớn.

- GPT-4o-mini: Mô hình Giám khảo, đóng vai trò thẩm định độc lập để so sánh hiệu quả giữa các ca sử dụng. Việc sử dụng một mô hình khác họ là chiến lược quan trọng để loại bỏ "thiên kiến tự thân" (self-preference bias) - hiện tượng LLM thường chấm điểm cao hơn cho văn bản do chính mình hoặc họ mô hình của mình sinh ra [4].

3.4 Kết quả thử nghiệm

Thực hiện thử nghiệm với 17 ca sử dụng trong bộ dữ liệu, với công thức:

$$S_{final} = 0.2 \cdot S_{sem} + 0.2 \cdot S_{ent} + 0.2 \cdot S_{fac} + 0.4 \cdot S_{str} \quad (3.1)$$

Trong đó:

- S_{final} : Điểm đánh giá trung bình cuối cùng.
- S_{sem} (Semantic Coverage): Điểm độ bao phủ ngữ nghĩa.
- S_{ent} (Entity Alignment): Điểm tương thích tác nhân.
- S_{fac} (Factuality): Điểm độ chính xác thực tế.
- S_{str} (Structure): Điểm cấu trúc và luồng sự kiện.

Mỗi phiên bản được đánh giá bởi LLM giám khảo 3 lần, rồi lấy kết quả trung bình của các hạng mục điểm. Điểm trung bình qua thử nghiệm với 17 ca sử dụng trong bộ dữ liệu có thể được biểu diễn như sau:

3.5 Nhận xét

Bảng 3.1 trình bày kết quả định lượng so sánh giữa hai phiên bản. Kết quả cho thấy phương pháp đề xuất mang lại sự cải thiện rõ rệt trên tất cả các tiêu chí, đặc biệt là ở Độ bao phủ luồng (tăng 14.7%).

Mặc dù phiên bản Cải thiện có xu hướng sinh ra chi tiết hơn dẫn đến nguy cơ dư thừa thông tin, nhưng chỉ số Độ chính xác về bước vẫn tăng từ 7.5 lên 8.6. Điều này chứng tỏ các bước được thêm vào phần lớn là các bước làm rõ logic nghiệp vụ (như xác nhận từ hệ thống, kiểm tra ngân sách) chứ không phải là ảo giác sai lệch.

Bảng 3.1: Kết quả đánh giá so sánh trung bình giữa Ca sử dụng Cơ sở và Ca sử dụng Cải thiện trên thang điểm 10

<i>Score</i>	Phiên bản Cơ sở	Phiên bản Cải thiện	Δ
S_{sem}	7.2	8.0	+11.1%
S_{ent}	8.5	8.9	+8.2%
S_{fac}	6.8	7.2	+5.8%
S_{str}	7.5	8.6	+14.7%
S_{final}	7.50	8.26	+10%

Với các dữ liệu ca sử dụng mà có mô tả ban đầu của người dùng rõ ràng về các bước, phiên bản Cơ sở cũng đã rất đầy đủ và sát với đáp án, khiến cho việc cải thiện không có quá nhiều sự khác biệt rõ rệt, ngược lại, còn gây ra hiện tượng tìm kiếm các bước không có trong mô tả. Mặt khác, với các mô tả đầu vào mơ hồ, phiên bản cải thiện lại cho thấy hiệu quả rất tốt trong việc khám phá và chỉnh sửa các nhánh mới. Điều này cho thấy bộ dữ liệu tự tổng hợp hiện tại vẫn còn hạn chế, cũng như sự cần thiết có chỉ dẫn thêm của con người để giúp LLM sinh được đúng hướng.

Chương 4

Kết luận

Kết quả thực nghiệm đã chứng minh tính khả thi và hiệu quả vượt trội của phương pháp tự động hóa kỹ thuật yêu cầu đã được đề xuất ở trên. Đặc biệt, việc tích hợp cơ chế tự sinh câu hỏi và kiểm chứng theo luồng đã giải quyết đáng kể các hạn chế cố hữu về tính ảo giác và thiên kiến nhất quán thường gặp trong các mô hình sinh văn bản.

Cụ thể, nghiên cứu đóng góp ba kết quả quan trọng sau:

- Đề xuất cấu trúc dữ liệu chuẩn hóa (GenUseCase): Khóa luận đã thiết kế và hiện thực hóa cấu trúc dữ liệu GenUseCase, đóng vai trò là định dạng biểu diễn trung gian thống nhất. Cấu trúc này không chỉ đảm bảo tính toàn vẹn của thông tin khi chuyển đổi từ ngôn ngữ tự nhiên sang đặc tả kỹ thuật, mà còn làm tiền đề vững chắc cho việc ánh xạ tự động sang các sơ đồ trực quan (PlantUML), giúp chuẩn hóa đầu vào và đầu ra cho toàn bộ hệ thống.
- Xây dựng phương pháp đánh giá định lượng: Dự án đã phát triển thành công hàm tính điểm định lượng dựa trên bộ tiêu chí kiểm tra và các quy tắc kinh nghiệm. Công cụ này đóng vai trò là thước đo khách quan, cho phép hệ thống tự động xác định chất lượng hiện tại của Ca sử dụng, từ đó định hướng chính xác các điểm cần cải thiện thay vì dựa vào cảm tính định tính.
- Hoàn thiện quy trình xác minh phân tách: Nghiên cứu đã cụ thể hóa phương pháp Chuỗi xác minh (CoVe) thành một quy trình nghiệp vụ rõ ràng. Bằng cách tách biệt tác vụ sinh câu hỏi phản biện và tác vụ giải quyết vấn đề thành các luồng độc lập, hệ thống đã triệt tiêu được sự can thiệp chéo giữa các ngữ cảnh, buộc mô hình phải tư duy đa chiều. Kết quả là các Ca sử dụng được sinh ra có độ chính xác cao hơn, logic chặt chẽ hơn và bám sát yêu cầu thực tế của người dùng.

Kết quả thực nghiệm cho thấy tầm quan trọng của việc xây dựng một bộ dữ liệu đạt chuẩn, đồng thời khẳng định tiềm năng phát triển của hệ thống theo hướng

tích hợp phản hồi từ người dùng. Thay vì tự động hóa hoàn toàn quy trình như hiện nay, việc kết hợp thêm ý kiến phản hồi của người dùng sẽ góp phần nâng cao hiệu quả và chất lượng của hệ thống.

Tài liệu tham khảo

- [1] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models, 2023.
- [2] Duc-Hanh Dang. FRSL: a domain specific language to specify functional requirements. *VNU Journal of Science Computer Science and Communication Engineering*, 6 2023.
- [3] Technological University of the Mixteca. Checklist: Use case.
- [4] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on LLM-as-a-Judge, 11 2024.